

Cómo usar las promesas

[< Anterior](#)
[Visión general: JavaScript asincrónico](#)
[Siguiente >](#)

Las promesas son la base de la programación asíncrona en el JavaScript moderno. Una promesa es un objeto devuelto por una función asincrónica, que representa el estado actual de la operación. En el momento en que la promesa se devuelve al llamante, la operación a menudo no está terminada, pero el objeto de promesa proporciona métodos para manejar el éxito o fracaso eventual de la operación.

Requisitos previos:	Un sólido conocimiento de los fundamentos de JavaScript y conceptos asincrónicos, como se ha tratado en lecciones anteriores de este módulo.
Resultados de aprendizaje:	- Los conceptos y fundamentos del uso de promesas en JavaScript. - Encadenando y combinando promesas. - Gestionar errores en promesas. <code>async</code> y <code>y</code>: cómo se relacionan con las promesas y por qué son útiles. <code>await</code>

En el [artículo anterior](#), hablamos sobre el uso de callbacks para implementar funciones asíncronas. Con ese diseño, llamas a la función asincrónica, pasando tu función de callback. La función regresa inmediatamente y llama a tu callback cuando la operación termina.

Con una API basada en promesas, la función asíncrona inicia la operación y devuelve un objeto **Promesa**. Luego puedes adjuntar manejadores a este objeto de promesa, y estos manejadores se ejecutarán cuando la operación haya tenido éxito o fracasado.

Usando la API `fetch()`

Nota: En este artículo, exploraremos las promesas copiando ejemplos de código de la página a la consola de JavaScript de tu navegador. Para plantearlo:

1. Abre una pestaña del navegador y visita <https://example.org>
2. en esa pestaña, abre la consola de JavaScript en [las herramientas de desarrollo de tu navegador](#)
3. Cuando mostremos un ejemplo, cópialo en la consola. Tendrás que recargar la página cada vez que introduzcas un nuevo ejemplo, o la consola se quejará de que has vuelto a declarar `.fetchPromise`

En este ejemplo, descargaremos el archivo JSON de <https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json> y registraremos algo de información sobre él.

Para ello, haremos una **petición HTTP** al servidor. En una solicitud HTTP, enviamos un mensaje de solicitud a un servidor remoto, y este nos responde. En este caso, enviaremos una solicitud para obtener un archivo JSON del servidor. ¿Recuerdas en el artículo anterior, donde hicimos solicitudes HTTP usando la API `XMLHttpRequest`? Bueno, en este artículo usaremos la `API fetch()`, que es el reemplazo moderno basado en promesas para `XMLHttpRequest`

Copia esto en la consola de JavaScript de tu navegador:

JS

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

console.log(fetchPromise);

fetchPromise.then((response) => {
  console.log(`Received response: ${response.status}`);
});

console.log("Started request...");
```

Aquí estamos:

1. llamando a la API y asignando el valor de retorno a la variable `fetch()` `fetchPromise`
2. inmediatamente después, registrando la variable. Esto debería dar algo como: `, indicándonos que tenemos un objeto, y que tiene un cuyo valor es . El estado significa que la operación de búsqueda sigue en marcha.` `fetchPromise` `Promise { <state>: "pending" }` `Promise state "pending" "pending"`
3. pasando una función manejadora al método `then()` de la Promesa. Cuando (y si) la operación de obtención tiene éxito, la promesa llamará a nuestro manejador, pasando un objeto `Respuesta`, que contiene la respuesta del servidor.
4. Registrando un mensaje indicando que hemos iniciado la solicitud.

La salida completa debería ser algo así:

```
Promise { <state>: "pending" }
Started request...
Received response: 200
```

Ten en cuenta que eso se registra antes de recibir la respuesta. A diferencia de una función síncrona, devuelve mientras la solicitud sigue en curso, permitiendo que nuestro programa siga respondiendo. La respuesta muestra el [código de estado](#) (OK), lo que significa que nuestra solicitud ha tenido éxito. `Started request... fetch() 200`

Esto probablemente se parezca mucho al ejemplo del artículo anterior, donde añadimos manejadores de eventos al objeto `XMLHttpRequest`. En lugar de eso, estamos pasando un manejador al método de la promesa devuelta. `then()`

Encadenando promesas

Con la API, una vez que obtienes un objeto, necesitas llamar a otra función para obtener los datos de respuesta. En este caso, queremos obtener los datos de respuesta como JSON, así que llamaríamos al método `json()` del objeto. Resulta que eso también es asíncronico. Así que este es un caso en el que tenemos que llamar a dos funciones asíncronas sucesivas. `fetch() Response Response json()`

Prueba esto:

JS

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise.then((response) => {
  const jsonPromise = response.json();
  jsonPromise.then((data) => {
    console.log(data[0].name);
  });
});
```

En este ejemplo, como antes, añadimos un manejador a la promesa devuelta por `fetch()`. Pero esta vez, nuestro manejador llama `response.json()`, y luego pasa un nuevo manejador a la promesa devuelta por `response.json().then()`.

Esto debería registrar "frijoles horneados" (el nombre del primer producto listado en "products.json").

¡Pero espera! ¿Recuerdas el último artículo, donde dijimos que al llamar a una callback dentro de otra callback, obtíamos niveles de código sucesivamente más anidados? ¿Y dijimos que este "infierno de referencia" hacía que nuestro código fuera difícil de entender. ¿No es lo mismo, solo que con llamadas `then()`?

Por supuesto que sí. Pero la característica elegante de las promesas es que a sí mismas devuelve una nueva promesa que se cumple con el valor de retorno de la función de llamada (siempre que la función se ejecute con éxito). Esto significa que podemos (y ciertamente deberíamos) reescribir el código anterior de esta manera: `then().then().json().then()`.

JS

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise
  .then((response) => response.json())
  .then((data) => {
    console.log(data[0].name);
  });
```

En lugar de llamar al segundo dentro del manejador para el primero, podemos *devolver* la promesa devuelta por `response.json()`, y llamar al segundo sobre ese valor de retorno. Esto se llama **encadenamiento de promesas** y significa que podemos evitar niveles cada vez mayores de sangría cuando necesitamos realizar llamadas consecutivas a funciones asíncronas. `then().then().json().then()`

Antes de pasar al siguiente paso, hay una pieza más que añadir. Tenemos que comprobar que el servidor ha aceptado y ha podido gestionar la solicitud antes de intentar leerla. Lo haremos comprobando el código de estado en la respuesta y lanzando un error si no estaba "OK":

JS

```
const fetchPromise = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    return response.json();
  })
  .then((data) => {
    console.log(data[0].name);
  });
```

Errores en la captura

Esto nos lleva a la última parte: ¿cómo gestionamos los errores? La API puede generar un error por muchas razones (por ejemplo, porque no había conectividad de red o porque la URL estaba mal formada de alguna manera) y nosotros mismos generamos un error si el servidor devolvió un error. `fetch()`

En el último artículo, vimos que el manejo de errores puede ser muy difícil con callbacks anidados, lo que nos obliga a manejar errores en todos los niveles de anidación.

Para soportar el manejo de errores, los objetos proporcionan un método `catch()`. Esto es muy parecido: lo llamas y pasas una función de manejador. Sin embargo, mientras que el manejador que se pasa a `then()` se llama cuando la operación *asíncrona tiene éxito*, el manejador que se pasa a `catch()` se llama cuando la operación *asíncrona falla*. `Promise then() then() catch()`

Si sumas al final de una cadena de promesas, entonces se llamará cuando falle alguna de las llamadas a funciones asíncronas. Así que puedes implementar una operación como varias llamadas consecutivas a funciones asíncronas, y tener un único lugar para manejar todos los errores. `catch()`

Prueba esta versión de nuestro código. Hemos añadido un gestor de errores usando `catch()`, y también hemos modificado la URL para que la solicitud falle. `fetch() catch()`

JS

```
const fetchPromise = fetch(
  "bad-scheme://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);

fetchPromise
  .then((response) => {
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
  })
```

```

    }
    return response.json();
  })
  .then((data) => {
    console.log(data[0].name);
  })
  .catch((error) => {
    console.error(`Could not get products: ${error}`);
  });

```

Prueba a ejecutar esta versión: deberías ver el error registrado por nuestro manejador. `catch()`

Terminología de promesas

Las promesas vienen acompañadas de una terminología bastante específica que merece la pena aclarar.

Primero, una promesa puede estar en uno de tres estados:

- **pendiente:** El estado inicial. La operación aún no se ha completado (ni tuvo éxito ni fracasó).
- **cumplida:** La operación tuvo éxito. Es entonces cuando llaman al encargado de la promesa. `.then()`
- **rechazado:** La operación fracasó. Es entonces cuando llaman al encargado de la promesa. `.catch()`

Ten en cuenta que lo que significa "exitoso" o "fallado" aquí depende de la API en cuestión. Por ejemplo, rechaza la promesa devuelta si (entre otras razones) un error de red impidió que se enviara la solicitud, pero cumple la promesa si el servidor envió una respuesta, incluso si la respuesta fue un error como **404 No encontrado**. `fetch()`

También usamos algunos otros términos para describir el estado de una promesa:

- **completado:** La promesa ya no está pendiente; O bien se ha cumplido o rechazado.
- **resuelto:** La promesa se cumple o ha sido "fijada" para seguir el estado de otra promesa. Este es un concepto más avanzado, relevante cuando una promesa depende de otra.

El artículo [Hablemos de cómo hablar de promesas](#)  ofrece una gran explicación de los detalles de esta terminología.

Combinar múltiples promesas

La cadena de promesas es lo que necesitas cuando tu operación consiste en varias funciones asíncronas, y necesitas que cada una se complete antes de comenzar la siguiente. Pero hay otras formas de combinar llamadas a funciones asíncronas, y la API proporciona algunos recursos para ellas. `Promise`

A veces, necesitas que todas las promesas se cumplan, pero no dependen unas de otras. En un caso así, es mucho más eficiente empezar todos juntos y luego recibir notificaciones cuando se hayan cumplido. El método `Promise.all()` es lo que necesitas aquí. Toma una serie de promesas y cumple una sola promesa.

La promesa que cumple es: `Promise.all()`

- **cumplido** cuando y si *todas* las promesas del arreglo se cumplen. En este caso, el manejador es llamado con un array de todas las respuestas, en el mismo orden en que se pasaron las promesas a `.then()` `all()`
- **rechazado** cuando y si *alguna* de las promesas del array es rechazada. En este caso, el manejador es llamado con el error lanzado por la promesa que rechazó. `catch()`

Por ejemplo:

JS

```
const fetchPromise1 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);
const fetchPromise2 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/not-found",
);
const fetchPromise3 = fetch(
  "https://mdn.github.io/learning-area/javascript/oajs/json/superheroes.json",
);

Promise.all([fetchPromise1, fetchPromise2, fetchPromise3])
  .then((responses) => {
    for (const response of responses) {
      console.log(`${response.url}: ${response.status}`);
    }
  })
  .catch((error) => {
    console.error(`Failed to fetch: ${error}`);
  });
```

Aquí, estamos haciendo tres peticiones a tres URLs diferentes. Si todos tienen éxito, registraremos el estado de respuesta de cada uno. Si alguno falla, entonces registramos el fallo. `fetch()`

Con las URLs que hemos proporcionado, todas las solicitudes deberían cumplirse, aunque para la segunda, el servidor devolverá (No encontrado) en lugar de (OK) porque el archivo solicitado no existe. Así que la salida debería ser: 404 200

```
https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json: 200
https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/not-found: 404
https://mdn.github.io/learning-area/javascript/oajs/json/superheroes.json: 200
```

Si probamos el mismo código con una URL mal formada, como esta:

JS

```
const fetchPromise1 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);
const fetchPromise2 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/not-found",
);
const fetchPromise3 = fetch(
  "bad-scheme://mdn.github.io/learning-area/javascript/oajs/json/superheroes.json",
);
```

```

Promise.all([fetchPromise1, fetchPromise2, fetchPromise3])
  .then((responses) => {
    for (const response of responses) {
      console.log(`${response.url}: ${response.status}`);
    }
  })
  .catch((error) => {
    console.error(`Failed to fetch: ${error}`);
  });

```

Entonces podemos esperar que el manejador ejecute, y deberíamos ver algo como: `catch()`

```
Failed to fetch: TypeError: Failed to fetch
```

A veces, puede que necesites que cualquiera de un conjunto de promesas se cumpla, y no te importe cuál. En ese caso, quieres `Promise.any()`. Esto es como `Promise.all()`, excepto que se cumple tan pronto como se cumple cualquiera de las promesas que se cumplen, o se rechaza si todas son rechazadas: `Promise.all()`

JS

```

const fetchPromise1 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
);
const fetchPromise2 = fetch(
  "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/not-found",
);
const fetchPromise3 = fetch(
  "https://mdn.github.io/learning-area/javascript/oojs/json/superheroes.json",
);

Promise.any([fetchPromise1, fetchPromise2, fetchPromise3])
  .then((response) => {
    console.log(`${response.url}: ${response.status}`);
  })
  .catch((error) => {
    console.error(`Failed to fetch: ${error}`);
  });

```

Ten en cuenta que en este caso no podemos predecir cuál petición de obtención se completará primero.

Estas son solo dos de las funciones extra para combinar múltiples promesas. Para saber más sobre el resto, consulta la documentación de [referencia de Promise](#). `Promise`

async y await

La palabra `clave async` te da una forma más sencilla de trabajar con código asíncrono basado en promesas. Añadir al inicio de una función la convierte en una función asíncrona: `async`

JS

```
async function myFunction() {  
  // This is an async function  
}
```

Dentro de una función asíncrona, puedes usar la palabra clave antes de una llamada a una función que devuelve una promesa. Esto hace que el código espere en ese momento hasta que se liquide la promesa, momento en el que el valor cumplido de la promesa se considera un valor de retorno, o se lanza el valor rechazado. `await`

Esto te permite escribir código que utiliza funciones asíncronas pero que parece código síncrono. Por ejemplo, podríamos usarlo para reescribir nuestro ejemplo de `fetch`:

JS

```
async function fetchProducts() {  
  try {  
    // after this line, our function will wait for the `fetch()` call to be settled  
    // the `fetch()` call will either return a Response or throw an error  
    const response = await fetch(  
      "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",  
    );  
    if (!response.ok) {  
      throw new Error(`HTTP error: ${response.status}`);  
    }  
    // after this line, our function will wait for the `response.json()` call to be settled  
    // the `response.json()` call will either return the parsed JSON object or throw an error  
    const data = await response.json();  
    console.log(data[0].name);  
  } catch (error) {  
    console.error(`Could not get products: ${error}`);  
  }  
}  
  
fetchProducts();
```

Aquí, estamos llamando , y en lugar de obtener un , nuestro llamador recibe de vuelta un objeto completamente completo, ¡como si fuera una función síncrona! `await fetch()` `Promise` `Response` `fetch()`

Incluso podemos usar un bloque para el manejo de errores, exactamente como haríamos si el código fuera síncrono. `try...catch`

Sin embargo, ten en cuenta que las funciones asíncronas siempre devuelven una promesa, así que no puedes hacer algo como:

JS

```
async function fetchProducts() {
  try {
    const response = await fetch(
      "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-
store/products.json",
    );
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(`Could not get products: ${error}`);
  }
}

const promise = fetchProducts();
console.log(promise[0].name); // "promise" is a Promise object, so this will not work
```



En su lugar, tendrías que hacer algo como:

JS

```
async function fetchProducts() {
  const response = await fetch(
    "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-
store/products.json",
  );
  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }
  const data = await response.json();
  return data;
}

const promise = fetchProducts();
promise
  .then((data) => {
    console.log(data[0].name);
  })
  .catch((error) => {
    console.error(`Could not get products: ${error}`);
  });
```

Aquí, movimos la parte trasera al manejador de la promesa devolta. Esto significa que nuestro manejador no tiene que lidiar con el caso en que se detecta un error dentro de la función, causando que sea `.data` `undefined`. Maneja los errores como el último paso de tu cadena de promesas. `try...catch` `catch` `then` `fetchProducts` `data` `undefined`

Además, ten en cuenta que solo puedes usarla dentro de una función, a menos que tu código esté en un [módulo de JavaScript](#). Eso significa que no puedes hacer esto en un guion normal: `await` `async`

JS

```
try {
  // using await outside an async function is only allowed in a module
  const response = await fetch(
    "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
  );
  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }
  const data = await response.json();
  console.log(data[0].name);
} catch (error) {
  console.error(`Could not get products: ${error}`);
  throw error;
}
```

Probablemente usarás muchas funciones donde de otro modo usarías cadenas de promesas, y hacen que trabajar con promesas sea mucho más intuitivo. `async`

Ten en cuenta que, al igual que una cadena de promesas, obliga a que las operaciones asíncronas se completen en serie. Esto es necesario si el resultado de la siguiente operación depende del resultado de la última, pero si no es así, algo como será más eficiente. `await` `Promise.all()`

Resumen

Las promesas son la base de la programación asíncrona en el JavaScript moderno. Facilitan expresar y razonar sobre secuencias de operaciones asíncronas sin callbacks profundamente anidados, y soportan un estilo de manejo de errores similar a la sentencia síncrona. `try...catch`

Las palabras clave y facilitan construir una operación a partir de una serie de llamadas consecutivas a funciones asíncronas, evitando la necesidad de crear cadenas de promesas explícitas y permitiéndote escribir código que se parezca exactamente a código síncrono. `async` `await`

Las promesas funcionan en las últimas versiones de todos los navegadores modernos; el único lugar donde el soporte de promesas será un problema es en Opera Mini, IE11 y versiones anteriores.

No hemos tocado todas las características de las promesas en este artículo, solo las más interesantes y útiles. A medida que empieces a aprender más sobre promesas, te encontrarás con más características y técnicas.

Muchas APIs web modernas se basan en promesas, incluyendo [WebRTC](#), [Web Audio API](#), [Media Capture y Streams API](#), y muchas más.

Véase también

- `Promise()`
- Usando promesas
- Tenemos un problema con las promesas [↗](#) de Nolan Lawson
- Hablemos de cómo hablar de promesas [↗](#)

[← Anterior](#)

[Visión general: JavaScript asíncronico](#)

[Siguiente →](#)



Tu plan para un internet mejor.